



UNIT 7

PROCESS MANAGEMENT AND SCHEDULING ALGORITHM

UNIT 7: PROCESS MANAGEMENT AND SCHEDULING ALGORITHM

Key Unit Competency: To be able to explain how processes are managed by Operating System and to understand process scheduling algorithms

INTRODUCTORY ACTIVITY

Open MS EXCEL, type your school name and save the file as "my school".

Press a combination of CTRL +ALT+ Delete.

Go to the Task Manager.

Click on Processes Tab and answer the following questions:

- What processes are running?
- Describe what is displayed in the table.
- Right Click on Microsoft EXCEL, Click on End Task. What do you observe? Is MS Excel still open or not? Why?

7.1 PROCESS

Learning activity 7.1.

Read the following text and answer the questions below.

There are two categories of programs. Application programs (usually called just "applications") are programs that people use to get their work done. Computers exist because people want to run these programs. System programs keep the hardware and software running together smoothly. The most important system program is the operating system. The operating system is always present when a computer is running. It coordinates the operation of the other hardware and software components of the computer system. The operating system is responsible for starting up application programs, running them, and managing the resources that they need. When an application program is running, the operating system manages the details of the hardware for it. For example, when you type characters on the keyboard, the operating system determines which application program they are intended for and does the work of getting them there.

- How do you call a program which is running in your computer?
- Through the interface obtained after combining CTRL+ALT+DELETE, which programs are running? What are their differences?
- Write down which ones are consuming a lot of memory space.

7.1.1. Definition

A process is a program in execution. It is an instance of program execution and it is not as same as program code but a lot more than it. A process is an 'active' entity as opposed to program which is considered to be a 'passive' entity. Attributes held by process include hardware state, memory, CPU etc.

The 4 sections in which the process is divided are:

- **Text:** This is the set of instructions, the compiled program code, read in from non-volatile storage, to be executed by the process.
- **Data:** this is the data used by the process when executing. It is made up the global and static variables, allocated and initialized prior to executing the main.
- **Heap of resources:** These are physical or virtual components of limited availability within a computer system that are needed by the process for its execution. They include CPU time, memory (random access memory as well as virtual memory), secondary storage like hard disks and external devices connected to the computer system. The heap is managed via calls to new, delete, malloc, free, etc.
- **Stack:** It is used for local variables. Space on the stack is reserved for local variables when they are declared.

The figure below shows the four sections of a process in memory for efficient working:

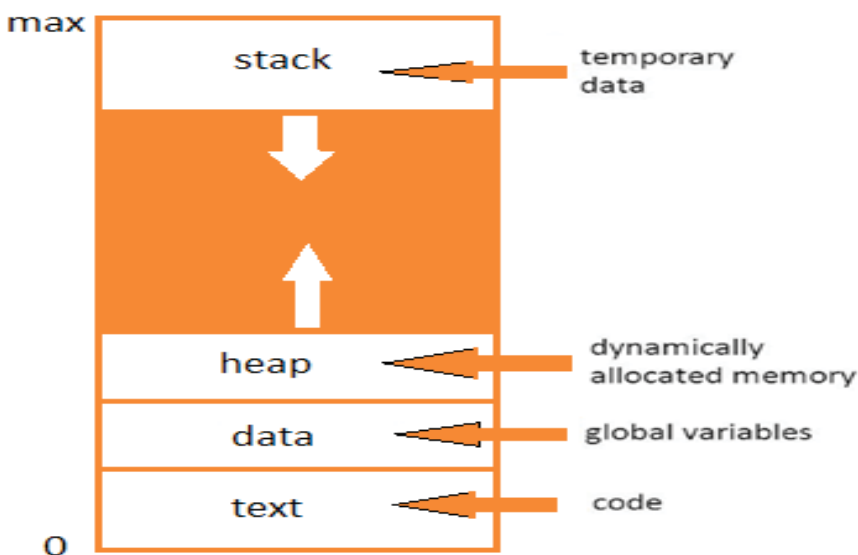


Figure 7.1.: A process in memory

7.1.2. Process execution requirement

A process in execution needs resources like the Central Processing Unit (CPU), memory and Input/output devices. In current computers, several processes share Resources where one processor is shared among many processes. The CPU and the input/output devices are considered as active resources of the system as they provide input and output services during a finite amount of time interval while the memory is considered as passive resource.

A process is executed by a processor through a set of instructions stored in memory. This instruction processing consists of two stages: The processor reads (fetches) instructions from the memory one after another and executes each instruction. Process execution consists of a number of repeated steps of fetching the process instruction and the instruction execution. The execution of a single instruction is called an instruction cycle. Graphically, it is shown by the figure below:

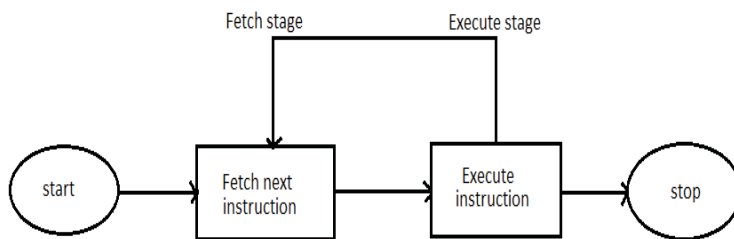


Figure 7.2. Process execution

7.1.3. Process vs. program

The major difference between a program and a process is that a program is a set of instructions to perform a designated task whereas the process is a program in execution. A process is an active entity because it involves some hardware state, memory, CPU etc. during its execution while a program is a passive entity because it resides in memory only. The differences between a program and a process are summarized in the table below:

Program	Process
A program consists of a set of instructions in a programming language	A process is a sequence of instructions in execution.
A program is a static object that exists in a file.	A process is a dynamic object i.e. a program in execution.
A program is in the secondary storage.	A process is loaded into the main memory.
The time span of a program is unlimited meaning that as long as the program is open it is considered to be running even though it is idle.	The time span of a process is limited.
A program is a passive entity which exists in the secondary storage persistently even if the machine reboots.	A process is an active entity since it is always stored in the main memory and disappears if the computer's power is switched off or has finished executing.

Table 1: differences between a program and a process

7.1.4. Process states

When a process executes, it changes the states whereas a process state is defined as the current activity of the process. A process can be in one of the following five states.

- New:** This is the initial state when a process is first created but it has not yet been added to the pool of executable processes.
- Ready:** At this state, the process is ready for execution and is waiting to be allocated to the processor. All ready processes are kept in a queue and they keep waiting for CPU time to be allocated by the Operating System in order to run. A program called scheduler picks up one ready process for execution.
- Running:** At this state, the process is now executing. The instructions within a process are executed. A running process possesses all the resources needed for its execution i.e. CPU time, memory (random access memory as well as virtual memory), secondary storage like hard disks and external devices connected to the computer system. As long as these resources are not available a process cannot go in the running state.
- Waiting:** a process is waiting for some event to occur before it can continue execution. The event may be a user input or waiting for a file to become available.
- Terminated:** This is when a process finishes its execution. A process

terminates when it finishes executing its last statement. When a process terminates, it releases all the resources it had and they become available for other processes.

So a process can be in one of these five states. Its model is also known as five state model and is shown below:

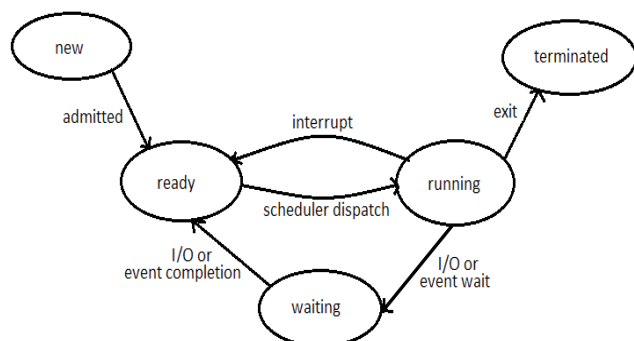


Figure 7.3: process states diagram

7.1.5. Process control block (PCB)

A Process Control Block is a data structure maintained by the Operating System for every process. It is also called Task Control Block and it is storage for information about processes.

When a process is created, the Operating System creates a corresponding PCB and when the process terminates, its PCB is released to the pool of free memory locations from which new PCBs are made. A process can compete for resources only when an active PCB is associated with it.

The table below shows the contents of a PCB with their meanings.

Contents	Meaning
Process ID	This is a unique process number that identifies each process
Process State	This indicates the current state of the process i.e., new, ready, running, waiting and terminated.
Pointer	This is a pointer to parent process. A parent process is a process that creates another process.

Program counter	Program Counter contains the address of the process next instruction to be executed.
CPU registers	These include the accumulator and index registers. Whenever a processor switches from one process to another, the information about the current status of the old process is saved in the register along with the program counter so that the process can be allowed to continue correctly afterwards.
I/O status information	This includes the names of I/O devices used by a process, list of open files and the information about allocation of peripheral devices to a process.
Accounting information	This includes the actual CPU time used in executing a process.
Processor scheduling details	It includes the priority of a process and the address to scheduling queues.

Table 2: Contents of a PCB with their meanings

APPLICATION ACTIVITY 7.1.

1. Do the following question
 - a. Describe what a process is? How does it differ from a program?
 - b. Describe in brief the structure of a Process Control Block.
 - c. Draw a labeled diagram for the process state transitions and explain the different process states.
 - d. What are the top 5 programs that are frequently used at your school?
2. Answer the following questions by true or false:
 - a. A process in the running state is currently being executed by the CPU.
 - b. The process control block (PCB) is a data structure that stores all information about a process.
 - c. CPU scheduling determines which programs are in memory.

7.2. THREAD

Learning Activity 7.2.

Visit the internet and make research and answer the following questions:

- What is a thread? How does it differ from a process?
- Explain the 2 types of threads

7.2.1. Definition of a Thread

A thread is the smallest unit of processing that can be performed in an Operating System. A thread is also called a lightweight process. Threads provide a way to improve application performance through parallelism.

In traditional operating systems, each process has an address space and a single thread of control. However, there are situations in which it is desirable to have multiple threads of control that share a single address space, but run in quasi-parallel, as though they were in fact separate processes.

Each thread belongs to exactly one process and no thread can exist outside a process. Threads provide a suitable foundation for parallel execution of applications on shared memory multiprocessors.

Multiple threads can be executed in parallel on many computer systems. This multithreading generally occurs by time slicing, wherein a single processor switches between different threads. For instance, there are some PCs that contain one processor core but you can run multiple programs at once, such as typing in a document editor while listening to music in an audio playback program; though the user experiences these things as simultaneous, in truth, the processor quickly switches back and forth between these separate processes.

The following figure shows the working of a single-threaded and a multithreaded process:

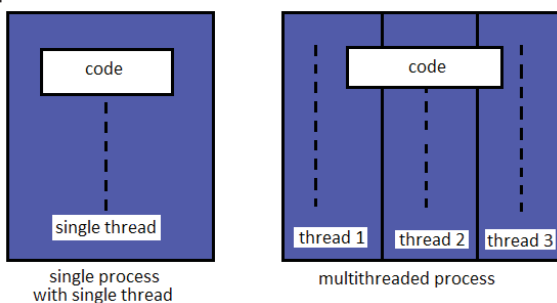


Figure 7.4: single and multithreaded processes

7.2.2 Difference between thread and process

A thread has some similarities to a process. Threads also have a life cycle, share the processor and execute sequentially within a process.

However there are differences as shown in the table below:

Process	Thread
Process cannot share the same memory	Threads can share memory and files.
Process creation is time consuming	Thread creation is not time consuming
Process execution is very slow	Thread execution is very fast
It takes more time to terminate a process	It takes less time to terminate a thread
It takes more time to switch between two processes.	It takes less time to switch between two threads.
Communication between processes is difficult	Communication between threads is easy and efficient.
It is a heavy weight process as it requires more resources.	It is a light weight process as it requires fewer resources.
They are not suitable for exploiting parallelism	They are suitable for exploiting parallelism
Process is bigger compared to a thread and one process may have many threads	A thread is smaller and is part of a process. Many related threads make up a process

Table3: Difference between thread and process

7.2.3 Types of threads

There are 2 types of threads namely User- level threads and Kernel level threads.

A. User - level threads

This type of thread is loaded in the user space only. The User space refers to all of the code in an operating system that lives outside of the kernel. The kernel does not know anything about them. When threads are managed in the user space, each process must have its own thread table. The thread table consists of the information

on the program counter and registers. This is shown in the figure 7.5a below:

Advantages of user - level threads:

- a. Each process can have its own process scheduling algorithm. This will be discussed later in the unit.
- b. User level threads can run on any Operating System.
- c. Faster switching among threads is possible.

Disadvantages of User-level threads

- a. When a user level thread executes a system call, not only that thread is blocked but also all the threads within the process are blocked. This is possible because the Operating System is unaware of the presence of threads and only knows about the existence of a process that constitutes these threads. A system call is a way for user programs to request some service from Operating System.
- b. Multithreaded application using user-level threads cannot take advantage of multiprocessing since the OS is unaware of the presence of threads and it schedules one process at a time.

B. Kernel-Level Threads

These are threads that are managed by the Operating System. There is no thread table in each process as the case of user level threads. The kernel only has the thread table. The thread table keeps track of all the threads in the system. The kernel's thread table holds each thread's registers and state. The information at the kernel level threads is the same as that at the user - level threads but the difference is that the information at the kernel - level is stored in the kernel space and at the user level thread; information is stored in the user space. Kernel space is part of the OS where the kernel executes and provides its services. This is shown in figure 7.5b below.

Advantages of Kernel level threads

- a. The OS is aware of the presence of threads in the processes. If one thread of a process gets blocked, the OS chooses the next one to run either from the same process or from a different one. A thread is blocked if it is waiting for an event, such as the completion of an I/O operation.
- b. It supports multiprocessing. The kernel can simultaneously schedule multiple threads from the same process on multiple processors

Disadvantages of Kernel level threads:

- a. The kernel-level threads are slow and inefficient. For instance, threads operations are hundreds of times slower than that of user-level threads.
- b. Since kernel must manage and schedule threads as well as processes. Switching between them is time consuming.

The figure below shows user level and kernel level threads

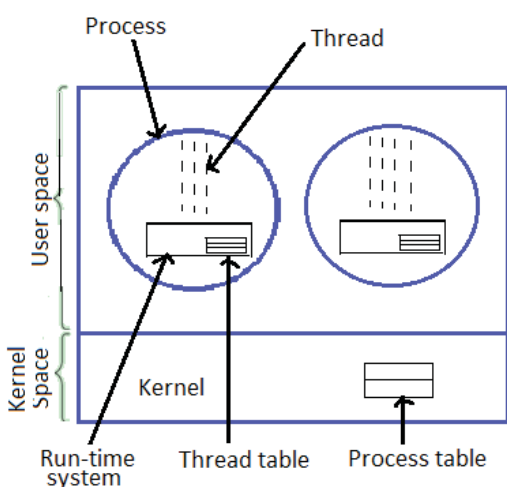


Figure 7.5a: User level thread

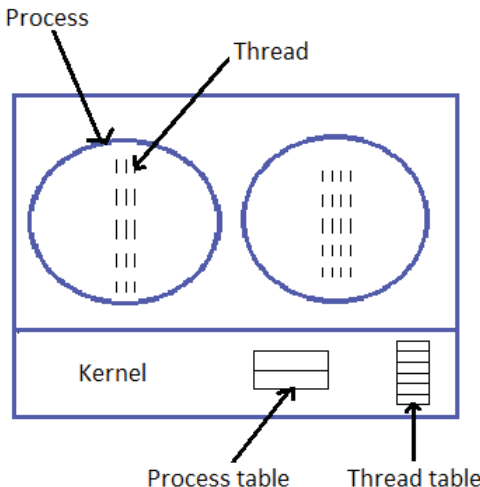


Figure 7.5b: Kernel level thread

Difference between User-Level and Kernel-Level Threads

Considering the above explanations of the 2 types of threads, the following table summarizes the differences between them.

User-Level Threads	Kernel-Level Threads
User-level threads are faster to create and manage.	Kernel-level threads are slower to create and manage.
Implementation is by a thread library at the user level.	Operating system supports creation of Kernel threads.
User-level thread is generic and can run on any operating system.	Kernel-level thread is specific to the operating system.
Multi-threaded applications cannot take advantage of multiprocessing.	Kernel level threads support multiprocessing.

Table 4: Difference between User-Level and Kernel-Level Threads

7.2.4 Advantages and disadvantages of threads

The advantages and disadvantages of threads are presented in the table below:

Advantages	Disadvantages
There is efficient communication because they do not need to use interprocess communication. Threads allow utilization of multiprocessor architectures to a greater scale and efficiency. They only need a stack and storage for registers therefore, threads are cheap to create. Threads use very little resources of an operating system in which they are working. That is, threads do not need new address space, global data, program code or operating system resources. Context switching is fast when working with threads. Context switching is the procedure of storing the state of an active thread or process for the CPU when it has to start executing a new one. Use of threads provides concurrency within a process.	Blocking: if the kernel is single threaded, a system call of one thread will block the whole process and CPU may be idle during the blocking period. Security: Since there is, an extensive sharing among threads there is a potential problem of security. It is quite possible that one thread over writes the stack of another thread (or damaged shared data) although it is very unlikely since threads are meant to cooperate on a single task.

Table 5: Advantages and disadvantages of threads

Application Activity 7.2.

Answer the following questions:

1. What are benefits of threads?
2. Discuss the advantages and the disadvantages of
 - a. User level thread
 - b. Kernel level thread

7.3. PROCESS SCHEDULING

Learning Activity 7.3.

During his working session on his computer in the School laboratory, the Laboratory Technician was doing many activities at the same time. By working with his text in Microsoft Word, he was also chatting with his friends on WhatsApp and also listening to his preferred Hip hop songs.

- i. What allowed him to do three activities at the same time on only one computer?

7.3.1 Definition of process scheduling

Process scheduling refers to the order in which the resources are allocated to different processes to be executed. Process scheduling is done by the process manager by removing running processes from the CPU and selects another process on the basis of a particular strategy.

7.3.2 Scheduling Queue

The scheduling queue is a queue that keeps all the processes in the Operating System. The Operating System maintains a separate queue for each of the process states and PCBs of all processes in the same execution state are placed in the same queue. When the state of a process is changed, its PCB is unlinked from its current queue and moved to its new state queue.

The Operating System maintains the following process scheduling queues:

- **Job queue:** this queue keeps all the processes in the system.
- **Ready queue:** this queue keeps a set of all processes residing in main memory, ready and waiting to be executed. A new process is always put in this queue.
- **Device queues:** the processes which are blocked due to unavailability of an I/O device constitute this queue.

The scheduling queues are shown in the figure below:

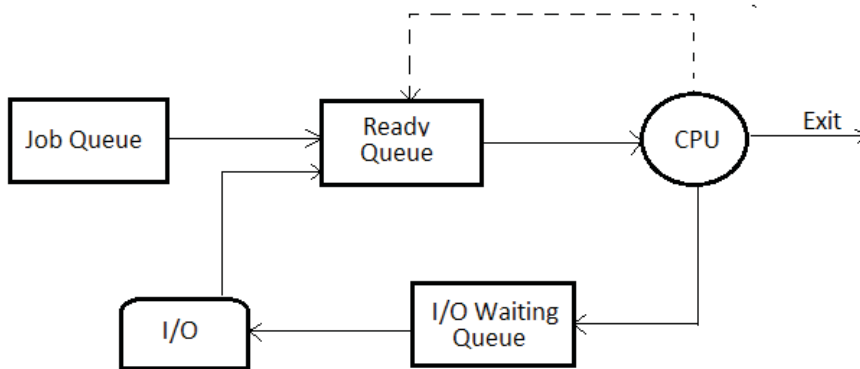


Figure 7.6: Process Scheduling Queues

All the processes entering into the system are put in a queue, called the **job queue**. The processes in the main memory that are ready and waiting for their chance to get executed are put in the queue called the **ready queue**.

It is common that a process undergoing execution may be interrupted temporarily, waiting for some other job to occur like completion of an I/O operation. All such waiting processes are put in a queue called a **device queue**.

A new process enters the ready queue and waits for its execution at the time it is allocated to a CPU.

7.3.4. Process State Model

A process can be in two main states namely running and not running. It is also called “the two process state model”. “The two process state model” is the model where two main states of the process i.e. running and not running are considered.

The two state process model is shown in the figure below:

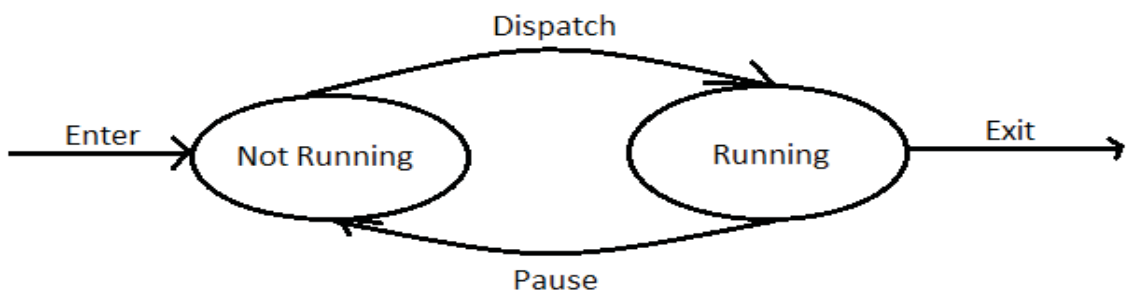


Figure 7.7: Two state process model

Suppose a new process P1 is created, then P1 is in Not Running state. When the CPU becomes free, the Dispatcher gives control of the CPU to P1 that is in Not Running state and waiting in a queue.

The Dispatcher is a program that gives control of the CPU to the process selected by the CPU scheduler.

When the dispatcher allows P1 to execute on the CPU, then P1 starts its execution. Therefore P1 is in running state. If a process P2 with high priority wants to execute on CPU, then P1 should be paused or P1 will be in the waiting state and P2 will be in the running state. When P2 terminates, then P1 again allows the dispatcher to execute it on the CPU.

However there can be abnormalities in the running of processes whereby they block one another. This situation is called **deadlock**.

7.3.4.1. Deadlock

A deadlock is a situation where a process or a set of processes are blocked, waiting for some resource that is held by some other waiting processes. Here a group of processes are permanently blocked as a result of each process having acquired a subset of the resources needed for its completion and waiting for the release of the remaining resource held by other processes in the same group. This makes it impossible for any of the processes to proceed.

A deadlock can be compared to a traffic jam illustrated below:



Figure 7.8: Traffic jam in Kigali City

Coffman (1971) identified four (4) conditions that must hold simultaneously for a deadlock to occur:

- a. **Mutual Exclusion:** a resource can be used by only one process at a time. If another process requests for that resource then the requesting process must be delayed until the resource becomes released.
- b. **Hold-and-Wait:** in this condition a process is holding a resource already allocated to it while waiting for an additional resource that is currently being held by other processes.
- c. **No preemption:** resources granted to a process can be released back to the system after the process has completed its task.
- d. **Circular wait:** the processes in the system form a circular list or chain where each process in the list is waiting for a resource held by the next process in the list. This is shown in the figure below:

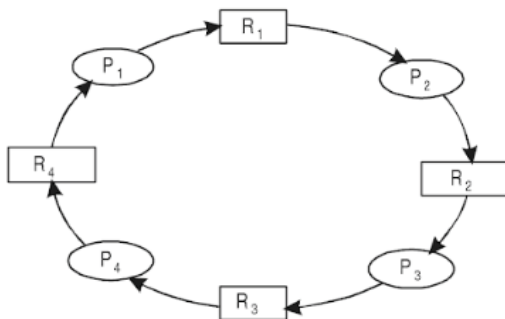


Figure 7.9.: Circular wait

7.3.5. Schedulers

A scheduler is an operating system program that selects the next job to be admitted for execution.

Schedulers are of three types namely: Long-Term Scheduler, Short-Term Scheduler and Medium-Term Scheduler.

a. Long-Term Scheduler

It is responsible for selecting the processes from secondary storage device like a disk and loads them into main memory for execution. It is also called a **job scheduler**. The reason why it is called a long term scheduler is because it executes less frequently as it takes a lot of time for the creation of new processes in the system.

b. Short-Term Scheduler

This scheduler allocates processes from the ready queue to the CPU for immediate processing. It is also called as **CPU scheduler**. Its main objective is to maximize CPU utilization. Short-term schedulers, also known as **dispatchers**. The dispatcher is the module that gives control of the CPU to the process selected by the short-term scheduler. Short-term schedulers are faster than long-term schedulers. They execute at least once every 10 millisecond.

c. Medium Term Scheduler

Medium-term scheduler removes the processes from the memory (swapping). A running process may become suspended if it makes an I/O request therefore it cannot make any progress towards completion. In order to remove the process from memory and make space for other processes, the suspended process is moved to the secondary storage. This process is called **swapping**, and the process is said to be swapped out or rolled out. The medium-term scheduler is in charge of handling the swapped out processes.

The working of the medium term scheduler is shown below:

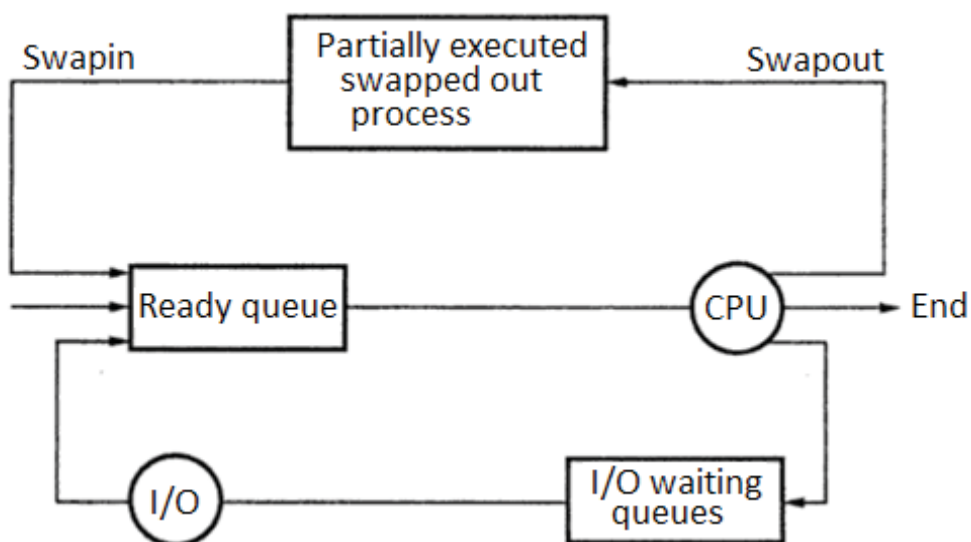


Figure 7.10.: Working of the medium term scheduler

The Comparison of the three different types of schedulers is summarized in the following table.

Long-Term Scheduler	Short-Term Scheduler	Medium-Term Scheduler
It is a job scheduler	It is a CPU scheduler	It is a process swapping scheduler.
Speed is lesser than short term scheduler	Speed is fastest among other two	Speed is medium in between both short and long term scheduler.
It controls the degree of multiprogramming	It provides lesser control over the degree of multiprogramming	It reduces the degree of multiprogramming.
Process state is (New to Ready)	Process state is (Ready to Running)	_____
It selects processes from pool and loads them into memory for execution	It selects those processes which are ready to execute.	It can re-introduce the process into memory and execution can be continued.

Table 6: Comparison of schedulers

Application Activity 7.3.

Choose the suitable word(s) in bold to fill the following blanks:

- An instance of a program execution is called _____ (**Process**, **Instruction**, **Procedure**, **Function**)
- A scheduler which selects processes from secondary storage device is called _____ (**Short term scheduler**, **Long term scheduler**, **Medium term scheduler**, **Process scheduler**).
- The _____ is used as the repository for any information of the process. (**Process state**, **deadlock**, **CPU**, **PCB**).
- _____ is the module that gives control of the CPU to the process selected by the short term scheduler. (**Dispatcher**, **processor**, **aging**).
- _____ Scheduler select the process that is in the ready queue to execute and allocate the CPU to it. (**Short term scheduler**, **Long term scheduler**, **Medium term scheduler**, **dispatcher**).

7.3.4. PROCESS SCHEDULING ALGORITHMS

Learning Activity 7.4.

1. Open multiple applications Microsoft WORD, Microsoft EXCEL and one used Browser. Observe on the Task Manager which one is coming first in the applications list? Explain why?
2. On the Performance Tab, click on Resource Monitor and explain why the order of processes available is changing dynamically?
3. What does the computer base on to place a process at the top and other in middle?

Process scheduling algorithms are procedures used by the job scheduler to plan for the different processes to be assigned to the CPU.

Objectives of Process Scheduling Algorithms

- Maximization of CPU utilization by keeping the CPU as busy as possible.
- Fair allocation of CPU to the processes.
- To maximize the number of processes that complete their execution per time unit. This is called throughput.
- To minimize the time taken by a process to finish its execution.
- To minimize the time a process waits in ready queue.
- To minimize the time it takes from when a request is submitted until the first response is produced.

There are 5 scheduling algorithms used by the process scheduler and they include the following:

- First-Come, First-Served (FCFS) Scheduling
- Shortest-Job-First (SJF) Scheduling
- Priority Scheduling
- Round Robin (RR) Scheduling
- Multiple-Level Queue Scheduling

The above mentioned algorithms are either **non preemptive or preemptive**. Non-preemptive algorithms means that once a process enters the running state, it cannot be preempted until it completes its allocated time. The preemptive scheduling means that a scheduler may preempt a low priority running process anytime when a high priority process enters into a ready state.

7.4.1. First Come, First-Served (FCFS) Scheduling

The basic principle of this algorithm is to allocate processes to the CPU in their order of arrival. FCFS is a non-preemptive scheduling algorithm. This algorithm is managed with a First in First out (FIFO) queue. It is non-preemptive because the CPU has been allocated to a process that keeps it busy until it is released.

When a process enters the ready queue, its process control block (PCB) is linked onto the tail of the queue. When the CPU is done executing a process and is free, the job scheduler selects from the head of the queue. This results into poor utilization of resources in the computer since it cannot utilize resources in parallel. When a set of processes need to use a resource for a short time and one process holds resources for a long time, it blocks all other processes leading to poor utilization of resources.

Key Terms:

- CPU Burst time: this is the time required to complete process execution in the CPU.
- Arrival time: this is the time at which the process arrives in the ready queue.
- Gantt chart: A Gantt chart is a horizontal bar chart illustrating process schedule.
- Completion time: This is the time at which process completes its execution.
- Wait Time: This is the time a process waits in the ready queue.
- Waiting Time = Completion time - Burst Time
- Turnaround Time: this is the total time between submission of a process and its completion.
- Turn Around Time = Completion Time - Arrival Time

For example consider the following set of processes with their arrival time and CPU burst time in milliseconds.

Process	CPU Burst Time (ms)	Arrival time
A	24	0
B	3	0
C	3	0

Case 1: When A, B and C arrive in the order A, B, C

The Gantt chart is shown below:

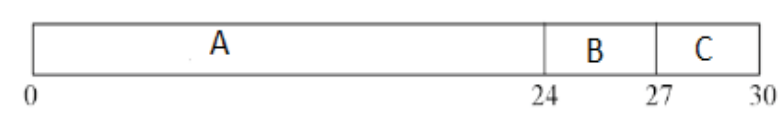


Figure 7.11a: Gantt chart FCFS algorithm

The wait time of each process is calculated as follows:

Wait Time: Completion Time –Burst time

Process	Wait Time
A	0
B	24
C	27

Average Wait Time (AWT)= $(0+24+27) / 3 = 17\text{ms}$

And Average Turnaround Time (ATAT) = $(24+27+30)/3 = 27\text{ms}$

Case 2: When A, B and C arrive in order B, C, A

The Gantt chart is as follows:

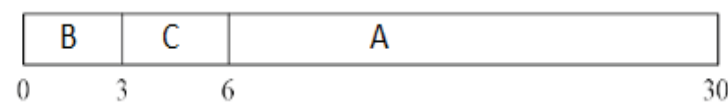


Figure 7.11b: Gantt chart FCFS algorithm

The wait time of each process is calculated as follows:

Wait Time: Completion time - Arrival Time

Process	Wait Time
A	6
B	0
C	3

Average Wait Time (AWT) = $(6+0+3) / 3 = 3\text{ms}$

And Average Turnaround Time (ATAT) = $(3+6+30)/3 = 13\text{ms}$

Case 2 is better than case 1 since AWT of case 2 is smaller than case 1. However the average waiting time under the FCFS algorithm is very long.

7.4.2 Shortest-Job-First (SJF) Scheduling

This is the algorithm that prioritizes the job whose execution time is smaller compared to other jobs already in the ready queue. When the CPU is free, it is assigned to the process of the ready queue which has the smallest CPU burst time. If two processes have the same CPU burst, the FCFS scheduling algorithm is used. SJF may either be preemptive or non preemptive.

Example: a) Non-preemptive SJF

Process	Burst time in ms	Arrival time
A	6	0
B	8	0
C	7	0
D	3	0

Assume the processes arrive in order A, B, C, D.
The Gantt chart is as follows:

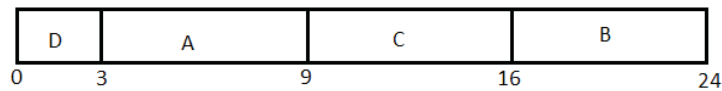


Figure 7.12a: Gantt chart for SJF non preemptive

Process	Wait Time
A	3
B	16
C	9
D	0

The average waiting time is $(0+3+9+16)/4 = 7\text{ms}$

b. Preemptive SJF

Process	CPU Burst	Arrival Time
A	10	0
B	2	2

Assume the processes arrive in the order A, B
The Gantt chart is shown below:

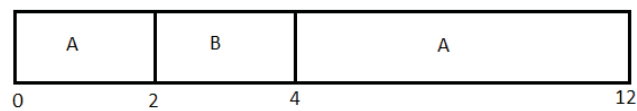


Figure 7.12b: Gantt chart for SJF preemptive

Process	Wait Time
A	4-2 =2
B	0

Average waiting time = (0+2)/2 = 1 ms

7.4.3. Priority Scheduling

In this algorithm, the CPU is allocated to the highest priority of the processes from the ready queue. Each process has a priority number. If two or more processes have the same priority, then FCFS algorithm is applied.

Priority scheduling can either be preemptive or non-preemptive. The choice is made whenever a new process enters the ready queue while some processes are running. If a newly arrived process has the higher priority than the current running process, the preemptive priority scheduling algorithm preempts the currently running process and allocates the CPU to the new process. On the other hand, the non-preemptive scheduling algorithm allows the currently running process to complete its execution and the new process has to wait for the CPU.

Consider four processes P1, P2, P3 and P4 with their arrival times, required CPU burst (in milliseconds) and priorities as shown in the following table:

Process	P1	P2	P3	P4
Arrival time	0	1	3	4
CPU burst (ms)	7	4	3	2
Priority	4	3	1	2

Assuming that the lower priority number means the higher priority, how will these processes be scheduled according to non-preemptive as well as preemptive priority

scheduling algorithm? Compute the average waiting time and average turnaround time in both cases.

a. Non- preemptive priority scheduling algorithm

The processes will be scheduled as shown in the following Gantt chart:

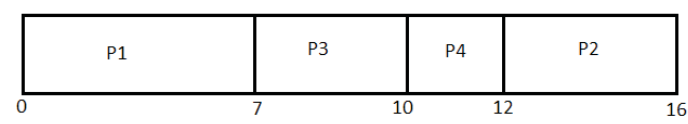


Figure 7.13a: Gantt chart for non-preemptive priority scheduling

Process	Wait Time
P1	0
P2	12-1 =11
P3	7-3=4
P4	10-4 = 6

Average waiting time =(0+11+4+6)/4 = 5.25ms

Average turnaround time = (7+15+7+8) /4= 9.25ms

b. Preemptive priority scheduling algorithm

The processes will be scheduled as shown in the Gantt chart below:

Process	P1	P2	P3	P4
Arrival time	0	1	3	4
CPU burst (ms)	7	4	3	2
Priority	4	3	1	2

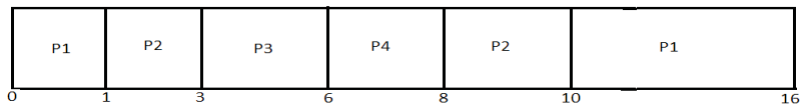


Figure 7.13b: Gantt chart for preemptive priority scheduling

Process	Wait Time
P1	10-1=9

P2	$8-2-1=5$
P3	0
P4	$6-4=2$

Average waiting time = $(9+5+0+2)/4 = 4\text{ms}$

Average turnaround time = $(16+9+3+4)/4 = 8\text{ms}$

A priority scheduling algorithm can make some low priority processes wait indefinitely for the CPU causing the problem called **starvation**. Starvation problem is solved by the **Aging** Technique whereby the priority of the processes waiting for a long time in the ready queue is increased.

7.4.4. Round Robin (RR) Scheduling

Round Robin scheduling algorithm is a preemptive algorithm. To implement RR scheduling, ready queue is maintained as a FIFO (First In First Out) queue of the processes. New processes are added to the tail of the ready queue. The CPU scheduler picks the first process from the ready queue and sets a timer to interrupt after 1 time quantum and dispatches the process.

With the RR algorithm, the length of the time quantum is very important. If it is very short, then short processes will be executed very quickly. If it is too large, the response time of the processes is too much which may not be tolerated in interactive environment. Response time is amount of time it takes from when a request is submitted until the first response is produced.

For example:

Process	CPU Burst Time
A	3
B	6
C	4
D	2

Time quantum is 2 milliseconds

The Gantt chart is as follows:

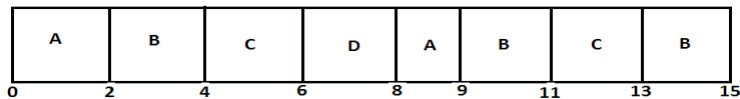


Figure 7.14.: Gantt chart for RR algorithm

The waiting time is calculated as follows:

Process	Waiting time
A	$0+6 = 6$
B	$2+5+2=9$
C	$4+5 = 9$
D	6

Average Wait Time= $(6+9+9+6) / 4 = 7.5\text{ms}$

7.4.5. Multiple-Level Queue Scheduling

In multilevel scheduling, all processes of the same priority are placed in a single queue. The figure 14 below shows the multilevel queue scheduling algorithm. It divides the ready queue into a number of separate queues. The processes are permanently assigned to one queue based on memory size, process priority and process type.

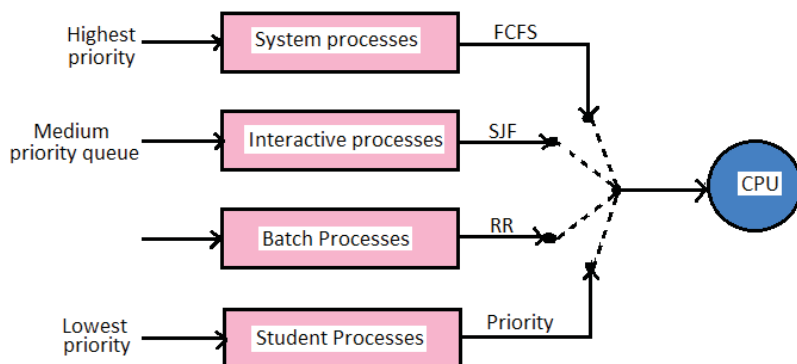


Figure 7.15: Multilevel Queue Scheduling

Each queue has its own scheduling algorithm. One queue may be scheduled by FCFS and another queue by RR method. Once the processes are assigned to the queue, they cannot move from one queue to another.

APPLICATION ACTIVITY 7.4.

Consider the following processes, with the CPU burst time given in milliseconds. The processes are arrived in P1, P2, P3, P4, P5 order of all at time 0.

Process	Burst time	Priority
P1	10	3
P2	1	1
P3	2	3
P4	1	4
P5	5	2

- c. Draw Gantt charts to show the execution using FCFS, SJF, non preemptive priority (small priority number implies higher priority) and RR (quantum =1) scheduling.
- d. What is the waiting time of each process for each one of the above scheduling algorithms?

END OF UNIT ASSESSMENT

Answer the following multiple choice questions.

1. An OS program module that selects the next job to be admitted for execution is called
 - a. Scheduler
 - b. Compiler
 - c. Throughput
 - d. None of the above
2. A short term scheduler executes at least once every
 - a. 1ms
 - b. 10ms
 - c. 5ms
 - d. None of the above
3. A program responsible for assigning the CPU to the process that has been selected by the short term scheduler is known as
 - a. Scheduler
 - b. Dispatcher
 - c. Debugger
 - d. None of the above
4. The indefinite blocking of low priority processes by a high priority process is known as
 - a. Starvation
 - b. Deadlock
 - c. Aging
 - d. None of the above
5. The technique of gradually increasing the priority of processes that wait in the system for a long time is called
 - a. Aging
 - b. Throughput
 - c. FCFS
 - d. None of the above
6. FIFO scheduling is
 - a. Preemptive
 - b. Non preemptive

c. Deadline scheduling

d. None of the above

7. A situation where a process or a set of processes is blocked, waiting for some resource that is held by some other waiting processes:
- Mutual exclusion
 - Hold and wait
 - Deadlock
 - None of the above

II. Answer the following structured questions:

- What do you understand by the following terms?
 - Process
 - Process state
 - Process control block
- Draw a labeled diagram for the process state transitions.
- Explain the following:
 - Short term scheduler**
 - Long term scheduler**
 - Medium term scheduler**
- Compare the process and program.
- What is the role of PCB? List the attributes of PCB.
- What are benefits of threads?
- What are the differences between user level threads and kernel supported threads?
- For the following example, calculate the average waiting time and average turnaround time for the following algorithms.
 - FCFS
 - Preemptive SJF
 - Round Robin (1 time unit)

Process	Arrival time	Burst time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

9. Consider the following set of processes

Process	Arrival time	Burst time
A	0	3
B	1	5
C	3	2
D	9	5
E	12	5

Draw the Gantt chart and calculate the waiting time and the average turnaround time for each of the following scheduling algorithms:

- a. FCFS
 - b. Non preemptive SJF
 - c. Preemptive SJF
10. Differentiate preemptive and non preemptive scheduling giving the application of each of them.
11. Suppose the following jobs arrive for processing at the times indicated. Each job will run the listed amount of time.

Job	Arrival time	Burst time
1	0.0	8
2	0.4	4
3	1.0	1

- a. Give the Gantt chart illustrating the execution of these jobs using the non-preemptive FCFS and SJF scheduling algorithm.
 - b. What is the wait time of each job for the above algorithm?
12. For the following set of processes, find the average waiting time using Gantt chart for
- a. SJF
 - b. Priority scheduling

Process	Burst time	Priority
P1	5	5
P2	3	4
P3	8	3
P4	2	1
P5	1	2

The processes have arrived in the order P2, P1, P4, P3 and P5.